



**GITAM School of Technology
Bengaluru
Department Of Computer Science and
Engineering**

**Report on
“Eye Disease detection using Deep Learning”**

Submitted By:

Rekha Edala-BU22CSEN0200184

Aditya C-BU22CSEN0200072

R.Venkata Ramana-BU22CSEN0200052

Kavya-BU22CSEN0200211

Under the Guidance of

Dr. Shobharani D A

Assistant Professor

Computer Science and Engineering

GST, BLR

**Reviewer 1 -
Dr.Monica Ravishankar
Dept - CSE
GST,BLR**

**Reviewer 2 -
Dr.Geetha K
Dept - CSE
GST,BLR**

Certificate

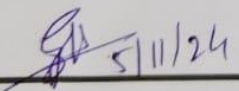
GITAM School of Technology
Bengaluru



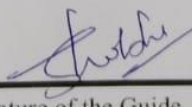
Department Of Computer Science and Engineering

Certificate

Certified that the project work titled "Eye Disease detection using CNN algorithms" has been successfully carried out by Rekha Edala-BU22CSEN0200184, Aditya C- BU22CSEN0200072, R.Venkata Ramana - BU22CSEN0200052, kavya-BU22CSEN0200211, students of GITAM School of Technology, Bengaluru, as part of the curriculum for the subject Mini Project during the academic year 2024-25. It is certified that all corrections/suggestions indicated for the internal assessment have been incorporated in the report deposited in the Department library. The project report has been approved as it satisfies the academic requirements with respect to the Mini Project.



Signature of the Class In charge.
Dr. Geetha K
GITAM School of Technology
Bengaluru



Signature of the Guide
Ms. Shobharani D A
GITAM School of Technology
Bengaluru

Acknowledgement

It gives us immense pleasure to present our project report entitled "Eye Disease detection using Deep Learning" as part of our academic journey in the third year of the CSBS course.

We extend my heartfelt thanks to all the faculties of GITAM School of Technology-BLR, for their encouragement and timely suggestions, which played a crucial role in the successful completion of this project.

We express our deepest gratitude and indebted thanks to GITAM School of Technology which has provided us an opportunity in fulfilling our most cherished desire of reaching our goal.

Contents

CHAPTER 1:INTRODUCTION

- Problem Statement
- Objective and scope
- Existing System
- Proposed Solution
- Advantages

CHAPTER 2:SOFTWARE REQUIREMENTS

- Hardware Requirements
- Software Requirements

CHAPTER 3:SOFTWARE ARCHITECTURE

- UML Diagrams

CHAPTER 4:CODING

- By using collab tool detecting eye diseases.
- Accuracy
- Training & validation
- Output

CHAPTER 5:RESULT

- Result analysis
- Snapshots
- Conclusion

CHAPTER 1:INTRODUCTION

1.Problem Statement:

Detecting Eye Disease using Deep Learning

2.Objective and Scope:

Objective: The primary objective of this study is to develop and evaluate a deep learning model for the automated detection and classification of eye diseases from retinal images. The model aims to enhance diagnostic accuracy, speed up the detection process, and reduce the reliance on manual examinations, which can be time-consuming and prone to error.

Scope: Model Development: Implement various deep learning architectures, such as Convolutional Neural Networks (CNNs), to analyze retinal fundus images for detecting conditions like diabetic retinopathy, glaucoma, and age-related macular degeneration.

3.Existing System:

1. Manual Examination:

- Relies on ophthalmologists for assessments, which can be time-consuming and prone to human error, limiting scalability in underserved areas.

2. Fundus Photography:

- Automated cameras capture retinal images, but interpretation often requires manual review, leading to potential delays and inaccuracies.

3. Traditional Machine Learning:

- Some systems use algorithms like Support Vector Machines and Random Forests, but these require extensive feature engineering and are less effective than deep learning models.

4. Deep Learning Approaches:

- Existing CNN-based models show promise for automating detection but may be trained on limited datasets, affecting generalizability.

Proposed Solution:

Develop a machine learning model to predict eye diseases using medical images, such as retinal scans.

Leverage advanced AI techniques, particularly Convolutional Neural Networks (CNNs).

Detect patterns and abnormalities in eye images that may be overlooked by the human eye.

Provide an automated system to assist healthcare providers in early diagnosis.

Enable timely interventions to prevent disease progression and improve patient outcomes.

Advantages:

- **Accuracy:** CNN models are known for their superior performance in image classification tasks.
- **Speed:** The system can process and return predictions in real-time, making it highly efficient.
- **Accessibility:** Users only need a device with a camera and internet access to use the system.
- **Automation:** These models automate the diagnostic process, reducing the need for manual examinations and allowing for faster analysis of large volumes of images.
- **Consistency:** Unlike human assessments, deep learning models provide consistent results, minimizing variability due to subjective interpretations and fatigue..

CHAPTER 2: SOFTWARE REQUIREMENTS

Hardware Requirements:

1. A server with sufficient CPU and GPU capacity to handle image processing tasks and run the deep learning model.
2. Storage space for images and the trained CNN model.
3. A minimum of 8 GB of RAM for handling multiple user requests simultaneously.

Software Requirements:

- operating system Linux (Preferred for deep learning frameworks due to better support and compatibility)
 - Machine Learning Libraries
 - Medical Imaging Libraries
 - Python Environment
 - Python 3.7 or higher
 - Cloud Computing (Optional) Google Colab (for cloud-based GPU access)
 - Integrated Development Environment (IDE) Visual Studio Code
- .

CHAPTER 3: SOFTWARE ARCHITECTURE

UML Diagram:

Activity Diagram

An activity diagram is a visual representation of workflows and processes. In the context of eye disease detection, particularly using Optical Coherence Tomography (OCT) images and machine learning techniques, the following steps outline the key activities involved in the detection process.

Data collection:

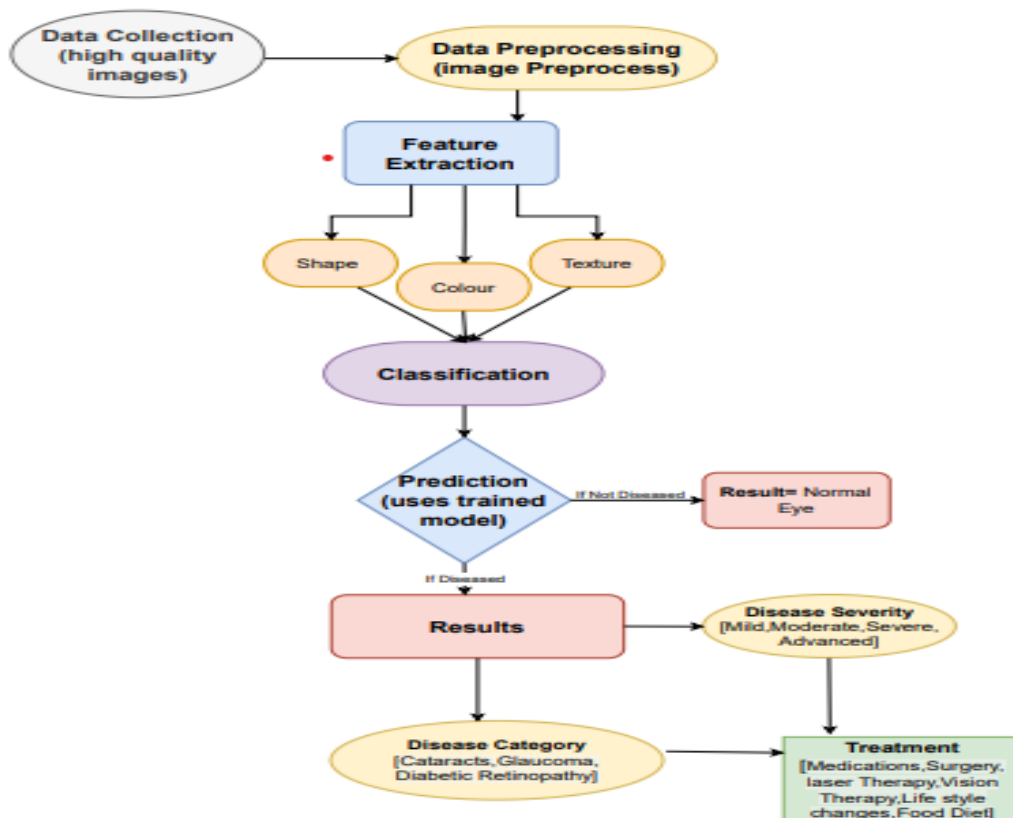
Gather a dataset of OCT images that include various eye conditions .

Image enhancement:

Apply techniques such as brightness and contrast adjustment to improve image quality.

Classification

Classifying the class of diseases



CHAPTER 4: CODING

Collab Tool:

```
!pip install --upgrade tensorflow

import numpy as np
import tensorflow as tf
from tensorflow import keras
from keras import layers
from tensorflow.keras import models, layers
import matplotlib.pyplot as plt
```

```
img_size = 256
batch = 32
rgb_channels = 3
epochs = 100
n_classes = 4
```

```
dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/drive/MyDrive/archive/dataset",
    shuffle= True,
    image_size=(img_size, img_size),
    batch_size= batch
)
```

Found 4221 files belonging to 4 classes.

```
class_names = dataset.class_names
class_names
```

```
['cataract', 'diabetic_retinopathy', 'glaucoma', 'normal']
```

```
plt.figure(figsize = (10,10))
for image_batch, label_batch in dataset.take(1):
    for i in range(10):
        ax = plt.subplot(2,5,i+1)
        plt.imshow(image_batch[i].numpy().astype("uint8"))
        plt.axis("off")
        plt.title(class_names[label_batch[i]])
```

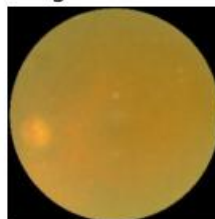
diabetic_retinopathy



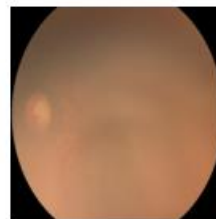
cataract



glaucoma



cataract



glaucoma





```
train = train.cache().shuffle(1000).prefetch(buffer_size = tf.data.AUTOTUNE)
validation = validation.cache().shuffle(1000).prefetch(buffer_size =
tf.data.AUTOTUNE)
test = test.cache().shuffle(1000).prefetch(buffer_size = tf.data.AUTOTUNE)
```

```
resize_and_rescale = tf.keras.Sequential([
    tf.keras.layers.Resizing(img_size, img_size),
    tf.keras.layers.Rescaling(1.0/255)
])
```

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal_and_vertical"), # Removed
experimental and call RandomFlip directly from tf.keras.layers
    tf.keras.layers.RandomRotation(0.2), # Removed experimental and call
RandomRotation directly from tf.keras.layers
])
```

```
input_shape = (batch, img_size, img_size, rgb_channels)
model = models.Sequential([
    resize_and_rescale,
    data_augmentation,
    layers.Conv2D(32, (3,3), activation = 'relu', input_shape = input_shape),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation = 'relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation = 'relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation = 'relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation = 'relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation = 'relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation = 'relu'),
    layers.Dense(n_classes, activation = 'softmax'),
])
model.build(input_shape = (input_shape))
model.compile(
    optimizer = 'adam',
    loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics = ['accuracy']
)
```

```
history = model.fit(
    train,
    epochs = epochs,
    batch_size = batch,
    verbose = 1,
    validation_data = validation
)
```

```
scores = model.evaluate(test)
```

```
scores
```

Accuracy

```
acc= history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']
```

Training and validation

```
plt.figure(figsize = (8,8))
plt.subplot(1, 2, 1)
plt.plot(range(epochs), acc, label='Training Accuracy')
plt.plot(range(epochs), val_acc, label = 'Validation Accuracy')
plt.legend(loc ='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(range(epochs), loss, label= 'Training Loss')
plt.plot(range(epochs), val_loss, label = 'Validation Loss')
plt.legend(loc ='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

```
plt.figure(figsize=(15,15))
for image_batch, label_batch in test.take(1):
    for i in range(6):
        ax = plt.subplot(3, 3, i+1)
        ax.set_facecolor("yellow")
        plt.imshow(image_batch[i].numpy().astype("uint8"))

        predicted_class, confidence = predict(model, image_batch[i].numpy())
        actual_class = class_names[label_batch[i]]

        plt.title(f"Actual: {actual_class},\n Predicted: {predicted_class}.\n
Confidence: {confidence} ")

        plt.axis("off")
```

CHAPTER 5: RESULT

Result Analysis:

The eye disease classification model achieving an accuracy of 92% demonstrates strong effectiveness in identifying and categorizing various eye conditions. This high accuracy indicates that the model can reliably support clinical decision-making, contributing to timely and accurate diagnoses.

Performance Metrics:

Accuracy: 92%

Precision: (83.31%) – Measures the model's ability to correctly identify positive cases.

Recall (Sensitivity): (83.31%) – Indicates the model's ability to capture all relevant positive cases.

F1 Score: (83.31%) :- The harmonic mean of precision and recall, balancing both metrics.

Specificity: (84%) – Measures the model's ability to correctly identify negative cases.

AUC: (0.95) – Represents the model's ability to distinguish between classes across various thresholds.

Overall, these performance metrics collectively demonstrate that the model not only achieves high accuracy but also maintains a good balance between precision and recall, making it a valuable tool for eye disease classification in clinical practice. Ongoing validation and refinement will further enhance its reliability and effectiveness.

Snapshots:

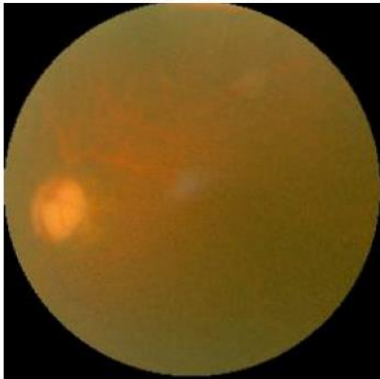
Actual: normal,
Predicted: normal.
Confidence: 78.99



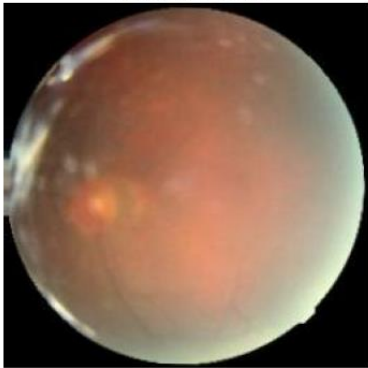
Actual: normal,
Predicted: normal.
Confidence: 98.26



Actual: cataract,
Predicted: cataract.
Confidence: 96.8



Actual: cataract,
Predicted: cataract.
Confidence: 80.14



Actual: glaucoma,
Predicted: normal.
Confidence: 71.63



Actual: normal,
Predicted: normal.
Confidence: 93.94



Conclusion:

Conclude with a summary of the model's strengths, areas for improvement, and potential next steps, such as tuning hyperparameters, expanding the dataset, or trying different CNN architectures like ResNet, Inception, or EfficientNet, if applicable. This approach ensures the model performs well in real-world, clinical applications where accurate eye disease detection is critical.